

Frequently asked questions

Status: Retired

ON THIS PAGE

Q: Why is SLSA not transitive?

Q: What about reproducible builds?

Q: How does SLSA relate to in-toto?

Q: What is the difference between a build platform, system, and service?

Q: Is SLSA the same as TACOS?

Q: How does SLSA and SLSA Provenance relate to SBOM?

Q: How to SLSA with a self-hosted runner

Q: Why is SLSA not transitive?

SLSA Build levels only cover the trustworthiness of a single build, with no requirements about the build levels of transitive dependencies. The reason for this is to make the problem tractable. If a SLSA Build level required dependencies to be the same level, then reaching a level would require starting at the very beginning of the supply chain and working forward. This is backwards, forcing us to work on the least risky component first and blocking any progress further downstream. By making each artifact's SLSA rating independent from one another, it allows parallel progress and prioritization based on risk.



in the case study [vision](#).

Q: What about reproducible builds?

When talking about [reproducible builds](#), there are two related but distinct concepts: “reproducible” and “verified reproducible.”

“Reproducible” means that repeating the build with the same inputs results in bit-for-bit identical output. This property [provides many benefits](#), including easier debugging, more confident cherry-pick releases, better build caching and storage efficiency, and accurate dependency tracking.

“Verified reproducible” means using two or more independent build platforms to corroborate the provenance of a build. In this way, one can create an overall platform that is more trustworthy than any of the individual components. This is often [suggested](#) as a solution to supply chain integrity. Indeed, this is one option to secure build steps of a supply chain. When designed correctly, such a platform can satisfy all of the SLSA Build level requirements.

That said, verified reproducible builds are not a complete solution to supply chain integrity, nor are they practical in all cases:

- Reproducible builds do not address source, dependency, or distribution threats.
- Reproducers must truly be independent, lest they all be susceptible to the same attack. For example, if all rebuilders run the same pipeline software, and that software has a vulnerability that can be triggered by sending a build request, then an attacker can compromise all rebuilders, violating the assumption above.
- Some builds cannot easily be made reproducible, as noted above.
- Closed-source reproducible builds require the code owner to either grant source access to multiple independent rebuilders, which is unacceptable in many cases, or develop multiple, independent in-house rebuilders, which is likely prohibitively expensive.

Therefore, SLSA does not require verified reproducible builds directly. Instead, verified reproducible builds are one option for implementing the requirements.

Q: How does SLSA relate to in-toto?

in-toto is a framework to secure software supply chains hosted at the **Cloud Native Computing Foundation**. The in-toto specification provides a generalized workflow to secure different steps in a software supply chain. The SLSA specification recommends **in-toto attestations** as the vehicle to express Provenance and other attributes of software supply chains. Thus, in-toto can be thought of as the unopinionated layer to express information pertaining to a software supply chain, and SLSA as the opinionated layer specifying exactly what information must be captured in in-toto metadata to achieve the guarantees of a particular level.

in-toto's official implementations written in **Go**, **Java**, and **Rust** include support for generating SLSA Provenance metadata. These APIs are used in other tools generating SLSA Provenance such as Sigstore's cosign, the SLSA GitHub Generator, and the in-toto Jenkins plugin.

Q. What is the difference between a build platform, system, and service?

Build platform and build system have been used interchangeably in the past. With the v1.0 specification, however, there has been a unification around the term platform as indicated in the **Terminology**. The use of the word **system** still exists related to software and services within the build platform and to systems outside of a build platform like change management systems.

A build service is a hosted build platform that is often run on shared infrastructure instead of individuals' machines and workstations. Its use has also been replaced outside of the requirements as it relates to the build platform.

Q: Is SLSA the same as TACOS?

No. **Trusted Attestation and Compliance for Open Source (TACOS)** is a framework authored by Tidelift. Per their website, TACOS is a framework "for assessing the development practices of open source projects against a set of secure development standards specified by the (US) NIST Secure



In contrast, SLSA is a community-developed framework—including adoptable guidelines for securing a software supply chain and mechanism to evaluate the trustworthiness of artifacts you consume—that is part of the Open Source Security Foundation (OpenSSF).

Q: How does SLSA and SLSA Provenance relate to SBOM?

Software Bill of Materials (SBOM) are a frequently recommended tool for increased software supply chain rigor. An SBOM is typically focused on understanding software in order to evaluate risk through known vulnerabilities and license compliance. These use-cases require fine-grained and timely data which can be refined to improve signal-to-noise ratio.

SLSA Provenance and the **Build track** are focused on trustworthiness of the build process. To improve trustworthiness, Provenance is generated in the build platform's trusted control plane, which in practice results in it being coarse grained. For example, in Provenance metadata completeness of `resolvedDependencies` information is on a best-effort basis. Further, the `ResourceDescriptor`

Version 1.1 is the current version. See [the Version 1.1 documentation](#).



While they likely include similar data, SBOMs and SLSA Provenance operate at different levels of abstraction. The fine-grained data in an SBOM typically describes the components present in a produced artifact, whereas SLSA Provenance more coarsely describes parameters of a build which are external to the build platform.

The granularity and expressiveness of the two use-cases differs enough that current SBOM formats were deemed not a good fit for the requirements of the Build track. Yet SBOMs are a good practice and may form part of a future SLSA Vulnerabilities track. Further, SLSA Provenance can increase the trustworthiness of an SBOM by describing how the SBOM was created.

SLSA Provenance, the wider **in-toto Attestation Framework** in which the recommended format sits, and the various SBOM standards, are all rapidly evolving spaces. There is ongoing investigation into linking between the different formats and exploration of alignment on common models. This FAQ entry describes our understanding of the intersection efforts today. We do not know how things will evolve

Q: How to SLSA with a self-hosted runner

Some CI systems allow producers to provide their own self-hosted runners as a build environment (e.g. [GitHub Actions](#)). While there are many valid reasons to leverage these, classifying the SLSA build level for the resulting artifact can be confusing.

Since the SLSA Build track describes increasing levels of trustworthiness and completeness in a package artifact's *provenance*, interpretation of the specification hinges on the platform entities involved in the provenance generation. The SLSA [build level requirements](#) (secure key storage, isolation, etc.) should be imposed on the transitive closure of the systems which are responsible for informing the provenance generated.

Some common situations may include:

- The platform generates the provenance and just calls a runner for individual build steps. In this situation, the provenance is only affected by the platform so there would be no requirements imposed on the runner.
- The runner generates the provenance. In this situation, the orchestrating platform is irrelevant and all requirements are imposed on the runner.
- The platform provides the runner with some credentials for generating the provenance or both the platform and the runner provide information for the provenance. Trust is shared between the platform and the runner so the requirements are imposed on both.

Additional requirements on the self-hosted runners may be added to Build levels greater than L3 when such levels get defined.

[◀ Guiding principles](#)[Future directions ▶](#)



information we collect does not identify anyone and does not track an individual's use of the site.

